PRESS 221

API de Gestion du Pressing

Documentation Technique Complète python

Version	1.0.0
Auteur	Cheikh IbrahimaThiam
Date	Avril 2025
Environnement	Python + PostgreSQL
Contact	digitalmaith@gmail.com
github	https://github.com/digitalmaith/press221_python.git

Présentation du projet (Version Python / FastAPI)

Contexte

PRESS 221 est une application de gestion destinée à digitaliser le fonctionnement d'un pressing. Elle permet de gérer efficacement les employés, les clients, les types de services proposés ainsi que les dépôts de vêtements effectués par les clients.

L'objectif est de centraliser toutes les opérations afin d'améliorer :

- le suivi des commandes
- la rapidité de traitement
- la traçabilité des vêtements déposés

Architecture API

PRESS 221 est une API RESTful développée avec **FastAPI**, respectant les principes de **Roy Fielding** :

- Architecture **stateless**
- Utilisation stricte des méthodes HTTP (GET, POST, PUT, DELETE)
- Ressources identifiées par des URI
- Échanges en JSON
- Codes HTTP normalisés

Périmètre fonctionnel

Employés

- CRUD complet
- Gestion des rôles :
    - REPASSEUR
    - NETTOYEUR
    - ACCUEIL

Clients

- CRUD complet
- Validation obligatoire :
  - email
  - téléphone

Types de service

- CRUD complet
- Contraintes :
  - code unique
  - prix > 0
  - délai (jours) > 0

Dépôts

- Enregistrement des dépôts
- Suivi des statuts (EN_ATTENTE, EN_COURS, TERMINÉ, RETIRÉ)
- Calcul automatique de la date de retrait prévue

Stack technologique (Version Python)

Technologie	Rôle
FastAPI	Framework web Python haute performance avec génération Swagger automatique
PostgreSQL	Base de données relationnelle — stockage de toutes les entités
SQLAlchemy 2.0	ORM Python — modèles orientés objet mappés aux tables
Pydantic v2	Validation des données entrantes et schémas de sortie
Alembic	Gestion des migrations de schéma de base de données
Uvicorn	Serveur ASGI pour exécuter l'application FastAPI

Installation et démarrage

2.1 Prérequis

- Python 3.12+
- PostgreSQL 14+
- pip (gestionnaire de paquets Python)

— Installation locale

```
# 2. Installer les dépendances
pip install -r requirements.txt

# 3. Configurer les variables d'environnement
cp .env.example .env
# Modifier DATABASE_URL dans .env selon votre config

# 4. Créer la base de données PostgreSQL
psql -U postgres -c "CREATE DATABASE press221;"

# 5. Démarrer l'API (tables créées automatiquement au lancement)
uvicorn main:app --reload
```

Architecture

```
press221/
│
├── app/
│   ├── api/
│   ├── core/
│   ├── models/
│   ├── db/
│   ├── schema/
│   ├── services/
│   └── init.py
│
├── alembic/
├── .env
├── requirements.txt
└── README.md
```

Modèle de Données (Les Entités)

model/client.py

```
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship

from app.db.session import Base


class Client(Base):
    __tablename__ = "clients"

    id = Column(Integer, primary_key=True, index=True)
    prenom = Column(String(100), nullable=False)
    nom = Column(String(100), nullable=False)
    telephone = Column(String(20), nullable=False)
    email = Column(String(150), unique=True, nullable=False,
index=True)
    adresse = Column(String(255))

    depots = relationship("Depot", back_populates="client")
```

Modèle `Client` — Explication du code

Ce code définit le modèle **Client** en utilisant l'ORM SQLAlchemy.
Il représente la table `clients` dans la base de données et permet de manipuler les données sous forme d'objets Python.

Importations

from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import relationship
from app.db.session import Base

- `Column`, `Integer`, `String` : permettent de définir les colonnes de la table
- `relationship` : permet de créer une relation entre tables
- `Base` : classe de base utilisée pour déclarer les modèles (fournie par SQLAlchemy)

Relation avec les dépôts

depots = relationship("Depot", back_populates="client")

Explication

- Définit une relation entre `Client` et `Depot`
- Un client peut avoir **plusieurs dépôts** (relation One-to-Many)

Détails

- `"Depot"` : nom du modèle lié (défini ailleurs)
- `back_populates="client"` :
  - crée une relation bidirectionnelle
  - dans le modèle `Depot`, on doit avoir :

Ce modèle permet de :

- Représenter un client dans la base de données
- Garantir l'unicité de l'email
- Imposer des champs obligatoires (nom, prénom, téléphone, email)
- Lier un client à plusieurs dépôts

model/depot.py

```
from sqlalchemy import Column, Integer, String, Date, ForeignKey, Enum
as SAEnum
from sqlalchemy.orm import relationship
import enum

from app.db.session import Base


class StatutDepot(str, enum.Enum):
    DEPOSE = "DEPOSE"
    EN_TRAITEMENT = "EN_TRAITEMENT"
    PRET = "PRET"
    RETIRE = "RETIRE"
```

```
class Depot(Base):
    __tablename__ = "depots"

    id = Column(Integer, primary_key=True, index=True)
    client_id = Column(Integer, ForeignKey("clients.id"),
nullable=False)
    employe_id = Column(Integer, ForeignKey("employes.id"),
nullable=False)
    service_id = Column(Integer, ForeignKey("types_service.id"),
nullable=False)
    description = Column(String(500), nullable=False)
    date_depot = Column(Date, nullable=False)
    date_retrait_prevue = Column(Date, nullable=False)
    statut = Column(SAEnum(StatutDepot), nullable=False,
default=StatutDepot.DEPOSE)

    client = relationship("Client", back_populates="depots")
    employe = relationship("Employe", back_populates="depots")
    service = relationship("TypeService", back_populates="depots")
```

model/employe.py

```
from sqlalchemy import Column, Integer, String, Enum as SAEnum
from sqlalchemy.orm import relationship
import enum

from app.db.session import Base


class PosteEnum(str, enum.Enum):
    repasseur = "repasseur"
    nettoyeur = "nettoyeur"
    accueil = "accueil"


class Employe(Base):
    __tablename__ = "employes"

    id = Column(Integer, primary_key=True, index=True)
```

```
    prenom = Column(String(100), nullable=False)
    nom = Column(String(100), nullable=False)
    email = Column(String(150), unique=True, nullable=False,
index=True)
    telephone = Column(String(20))
    poste = Column(SAEnum(PosteEnum), nullable=False)

    depots = relationship("Depot", back_populates="employe")
```

model/typeService.py

```
from sqlalchemy import Column, Integer, String, Float
from sqlalchemy.orm import relationship

from app.db.session import Base


class TypeService(Base):
    __tablename__ = "types_service"

    id = Column(Integer, primary_key=True, index=True)
    code = Column(String(50), unique=True, nullable=False, index=True)
    libelle = Column(String(150), nullable=False)
    prix = Column(Float, nullable=False)
    delai = Column(Integer, nullable=False)  # en heures

    depots = relationship("Depot", back_populates="service")
```

Les services

clientService.py

```
from sqlalchemy.orm import Session
from fastapi import HTTPException, status
```

```python
from app.models.client import Client
from app.models.depot import Depot, StatutDepot
from app.schemas.client import ClientCreate, ClientUpdate


class ClientService:

    @staticmethod
    def get_all(db: Session) -> list[Client]:
        return db.query(Client).all()

    @staticmethod
    def get_by_id(db: Session, client_id: int) -> Client:
        client = db.query(Client).filter(Client.id ==
client_id).first()
        if not client:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                                detail=f"Client #{client_id}
introuvable.")
        return client

    @staticmethod
    def create(db: Session, data: ClientCreate) -> Client:
        if db.query(Client).filter(Client.email == data.email).first():
            raise HTTPException(status_code=status.HTTP_409_CONFLICT,
                                detail=f"L'email '{data.email}' est
déjà utilisé.")
        client = Client(**data.model_dump())
        db.add(client)
        db.commit()
        db.refresh(client)
        return client

    @staticmethod
    def update(db: Session, client_id: int, data: ClientUpdate) ->
Client:
        client = ClientService.get_by_id(db, client_id)
        for field, value in data.model_dump(exclude_none=True).items():
            setattr(client, field, value)
        db.commit()
```

```
        db.refresh(client)
        return client

    @staticmethod
    def delete(db: Session, client_id: int) -> None:
        client = ClientService.get_by_id(db, client_id)
        # Interdire suppression si dépôts non retirés
        statuts_actifs = [StatutDepot.DEPOSE,
StatutDepot.EN_TRAITEMENT, StatutDepot.PRET]
        depots_non_retires = (db.query(Depot)
                              .filter(Depot.client_id == client_id,
                                      Depot.statut.in_(statuts_actifs))
                              .count())
        if depots_non_retires > 0:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail=f"Impossible de supprimer : le client a
{depots_non_retires} dépôt(s) non retiré(s)."
            )
        db.delete(client)
        db.commit()
```

employeService.py

```
from sqlalchemy.orm import Session
from fastapi import HTTPException, status

from app.models.employe import Employe
from app.models.depot import Depot, StatutDepot
from app.schemas.employe import EmployeCreate, EmployeUpdate


class EmployeService:

    @staticmethod
    def get_all(db: Session) -> list[Employe]:
        return db.query(Employe).all()

    @staticmethod
    def get_by_id(db: Session, employe_id: int) -> Employe:
```

```
        employe = db.query(Employe).filter(Employe.id ==
employe_id).first()
        if not employe:
            raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,
                                detail=f"Employé #{employe_id}
introuvable.")
        return employe

    @staticmethod
    def create(db: Session, data: EmployeCreate) -> Employe:
        # Unicité email
        if db.query(Employe).filter(Employe.email ==
data.email).first():
            raise HTTPException(status_code=status.HTTP_409_CONFLICT,
                                detail=f"L'email '{data.email}' est
déjà utilisé.")
        employe = Employe(**data.model_dump())
        db.add(employe)
        db.commit()
        db.refresh(employe)
        return employe

    @staticmethod
    def update(db: Session, employe_id: int, data: EmployeUpdate) ->
Employe:
        employe = EmployeService.get_by_id(db, employe_id)
        for field, value in data.model_dump(exclude_none=True).items():
            setattr(employe, field, value)
        db.commit()
        db.refresh(employe)
        return employe

    @staticmethod
    def delete(db: Session, employe_id: int) -> None:
        employe = EmployeService.get_by_id(db, employe_id)
        # Interdire suppression si dépôts en cours
        statuts_actifs = [StatutDepot.DEPOSE,
StatutDepot.EN_TRAITEMENT, StatutDepot.PRET]
        depots_en_cours = (db.query(Depot)
                           .filter(Depot.employe_id == employe_id,
                                   Depot.statut.in_(statuts_actifs))
```

```
                            .count())
        if depots_en_cours > 0:
            raise HTTPException(
                status_code=status.HTTP_409_CONFLICT,
                detail=f"Impossible de supprimer : l'employé a
{depots_en_cours} dépôt(s) en cours."
            )
        db.delete(employe)
        db.commit()
```

Ce fichier implémente une couche **service métier** pour la gestion des employés.
Il utilise FastAPI pour la gestion des erreurs HTTP et SQLAlchemy pour l'accès à la base de données.

Le rôle principal de ce service est de :

- centraliser la logique métier
- éviter de surcharger les routes (controllers)
- garantir les règles de gestion

`Session` : permet d'interagir avec la base de données
`HTTPException` : permet de retourner des erreurs HTTP propres
`status` : fournit les codes HTTP standards (404, 409, etc.)
`Employe` : modèle ORM représentant la table employés
`Depot` : modèle des dépôts
`StatutDepot` : énumération des statuts possibles d'un dépôt
`EmployeCreate`, `EmployeUpdate` : schémas de validation (Pydantic)

Résumé des règles métier

- Email unique obligatoire
- Impossible de supprimer un employé avec des dépôts en cours
- Mise à jour partielle autorisée
- Gestion propre des erreurs HTTP

`EmployeService` joue un rôle clé dans l'architecture :

Il garantit que toutes les opérations sur les employés respectent les règles du système PRESS 221, tout en gardant un code propre, maintenable et évolutif.

schémas

```python
from pydantic import BaseModel, EmailStr, field_validator
from typing import Optional


class ClientBase(BaseModel):
    prenom: str
    nom: str
    telephone: str
    email: EmailStr
    adresse: Optional[str] = None

    @field_validator("prenom", "nom", "telephone")
    @classmethod
    def not_empty(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("Ce champ ne peut pas être vide.")
        return v.strip()


class ClientCreate(ClientBase):
    pass


class ClientUpdate(BaseModel):
    prenom: Optional[str] = None
    nom: Optional[str] = None
    telephone: Optional[str] = None
    adresse: Optional[str] = None


class ClientOut(ClientBase):
    id: int

    model_config = {"from_attributes": True}
```

Ce fichier définit les **schémas de validation et de sérialisation** pour l'entité Client en utilisant Pydantic.

Ces schémas servent à :

- valider les données entrantes (requêtes HTTP)
- structurer les réponses de l'API
- garantir la cohérence des données

Explication

- `BaseModel` : classe de base pour créer des schémas
- `EmailStr` : type spécial pour valider les emails
- `field_validator` : permet de définir des règles de validation personnalisées
- `Optional` : indique qu'un champ peut être absent ou nul

Explication

- Cette fonction s'applique aux champs :
  - `prenom`
  - `nom`
  - `telephone`

Règles appliquées

- Supprime les espaces inutiles (`strip()`)
- Vérifie que la valeur n'est pas vide
- Si vide → lève une erreur de validation

Schéma de création : `ClientCreate`

class ClientCreate(ClientBase):

  pass

Explication

- Hérite de `ClientBase`
- Utilisé pour les requêtes **POST**
- Aucun champ supplémentaire nécessaire

Ces schémas garantissent que toutes les données liées aux clients dans PRESS 221 sont :

- valides

- propres (sans espaces inutiles)
- cohérentes

Ils constituent une couche essentielle entre l'utilisateur et la base de données.

Db

[session.py]

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base, sessionmaker

from app.core.config import settings

engine = create_engine(settings.DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
bind=engine)
Base = declarative_base()


def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Configuration de la base de données — Explication du code

Ce fichier met en place la **connexion à la base de données** et fournit un mécanisme pour gérer les sessions avec SQLAlchemy dans une application FastAPI.

Il constitue le cœur de l'accès aux données dans ton projet PRESS 221.

Explication

- `create_engine` : crée la connexion à la base de données
- `declarative_base` : permet de définir les modèles ORM
- `sessionmaker` : fabrique des sessions pour interagir avec la base
- `settings` : contient les variables d'environnement (ex : URL de la DB)

Explication

- Initialise la connexion à la base de données
- `DATABASE_URL` contient une URL comme :

postgresql://user:password@localhost:5432/press221

Le moteur (`engine`) est responsable de :

- gérer les connexions
- exécuter les requêtes SQL

Explication

- `sessionmaker` crée une **fabrique de sessions**
- Chaque session représente une interaction avec la base

Paramètres importants

- `autocommit=False`
  → les changements doivent être validés manuellement avec `commit()`
- `autoflush=False`
  → évite l'envoi automatique des requêtes avant un commit
- `bind=engine`
  → lie la session au moteur de base de données

Ce fichier est essentiel pour :

- connecter l'application à la base de données
- gérer efficacement les sessions
- garantir la stabilité et les performances

Sans cette configuration, aucune interaction avec la base ne serait possible dans PRESS 221.

Règles de suppression sécurisée

EMPLOYE — Suppression bloquée (409 Conflict) si :

L'employé a des dépôts avec statut DEPOSE, EN_TRAITEMENT ou PRET.
Message : 'Impossible de supprimer : l'employé a X dépôt(s) en cours.'

CLIENT — Suppression bloquée (409 Conflict) si :

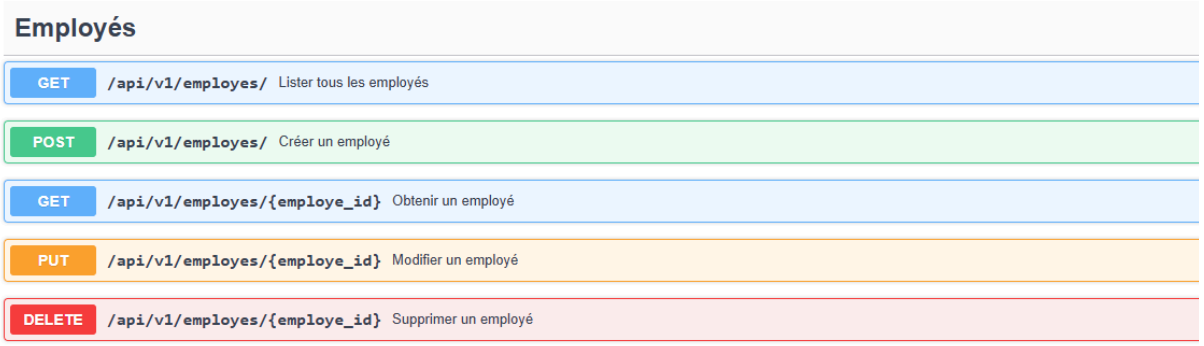
Le client a des dépôts avec statut DEPOSE, EN_TRAITEMENT ou PRET.
Message : 'Impossible de supprimer : le client a X dépôt(s) non retiré(s).'

TYPE DE SERVICE — Suppression bloquée (409 Conflict) si :

Au moins un dépôt (quel que soit son statut) utilise ce service.
Message : 'Impossible de supprimer : X dépôt(s) utilisent ce service.'

Swagger

lien : http://localhost:8000/docs



Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/api/v1/employes/' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/api/v1/employes/
```

Server response

Code	Details
200	Response body

```
[
  {
    "prenom": "string",
    "nom": "string",
    "email": "user@example.com",
    "telephone": "string",
    "poste": "repasseur",
    "id": 1
  }
]
```

Response headers

Clients

GET	/api/v1/clients/	Lister tous les clients
POST	/api/v1/clients/	Créer un client
GET	/api/v1/clients/{client_id}	Obtenir un client
PUT	/api/v1/clients/{client_id}	Modifier un client
DELETE	/api/v1/clients/{client_id}	Supprimer un client

Types de service

GET	/api/v1/services/	Lister tous les types de service
POST	/api/v1/services/	Créer un type de service
GET	/api/v1/services/{service_id}	Obtenir un type de service
PUT	/api/v1/services/{service_id}	Modifier un type de service
DELETE	/api/v1/services/{service_id}	Supprimer un type de service